TED UNIVERSITY

Faculty of Engineering

Department of Computer Engineering

Project Name: F&D Autonomous Driving

Test Plan Report

Team Members:

Can Soysal

Çağrı Şanlı

Mustafa Cem Hür

Taha Ugan

Taylan Kayalı

Project Supervisor:

Tolga Kurtuluş Çapın

Project Jury:

Deniz Cantürk

Hakkı Gökhan İlk

Project Website:

[Autonomous Driving Algorithm - Final Project]

Table Of Contents

1. Introduction....2
    1.1 Purpose....2
    1.2 Scope....2
    1.3 Objectives....2
    1.4 Definitions, Acronyms, and Abbreviations....2
2. Features to Be Tested....3
    2.1 Perception Subsystem (fd_perception)....3
    2.2 Planning Subsystem (fd_planning)....3
    2.3 Control Subsystem (fd_control)....3
    2.4 Simulation Interface (fd_simulation)....3
    2.5 End-to-End Scenarios....4
    2.6 Features NOT to Be Tested....4
3. Testing Methodology....4
    3.1 Unit Testing....4
    3.2 Integration Testing....4
    3.3 System Testing....5
    3.4 Performance Testing....5
    3.5 Regression Testing....5
    3.6 User Acceptance Testing (UAT)....5
4. Test Environment....6
    Test Data....6
5. Test Cases....6
    5.1 Perception Unit Tests (PER-U)....6
    5.2 Planning Unit Tests (PLN-U)....7
    5.3 Control Unit Tests (CTL-U)....8
    5.4 Simulation Unit Tests (SIM-U)....9
    5.5 Integration Tests (INT)....10
    5.6 System Tests (SYS)....11
    5.7 Performance Tests (PERF)....11
6. Test Schedule....12
7. Roles and Responsibilities....13
8. Control Procedures....13
    8.1 Defect Management....13
    8.2 Test Execution and Logging....13
    8.3 Pass/Fail Criteria....13
9. Risks and Mitigations....14
10. References....15

1. Introduction

1.1 Purpose

This Test Plan Report explains how the F&D Autonomous Driving System will be tested. It includes the testing strategy, scope, schedule, and team responsibilities. F&D is a ROS2-based virtual autonomous driving simulation built on Gazebo, and it is used to safely develop self-driving algorithms in a simulated environment. This document explains how each subsystem - Perception, Planning, Control, and Simulation - will be tested to make sure they work correctly, reliably, and efficiently.

1.2 Scope

Testing includes all main software components of the F&D system across four ROS2 packages:

- fd_perception: VisionProcessor, ObstacleClassifier, TrafficSignRecognizer, DepthEstimator, LocalizationEstimator
- fd_planning: RoadGraph, GlobalPlanner, LocalPlanner, BehaviorManager
- fd_control: TrajectoryTracker, EmergencyStop, ParkingController
- fd_simulation: GazeboSensorBridge, GazeboActuatorBridge, SimulationController

The shared message package fd_msgs is tested indirectly through integration tests. The launch package fd_launch is checked through end-to-end system tests.

1.3 Objectives

- Check that each class and method works according to its documented pre- and post-conditions in the LLD.
- Make sure subsystems communicate correctly using ROS2 topics and services.
- Confirm that the system meets the performance requirements (such as, >= 10 FPS pipeline; global plan within 10 s; local plan update <= 3 s).
- Ensure that safety features (EmergencyStop, stop-sign compliance) work reliably.

1.4 Definitions, Acronyms, and Abbreviations

CI	Continuous Integration
DWA	Dynamic Window Approach (local planning algorithm)
FPS	Frames Per Second
MPC	Model Predictive Control
PID	Proportional-Integral-Derivative control
ROS2	Robot Operating System 2
SUT	System Under Test
TC	Test Case
UAT	User Acceptance Testing
YOLO	You Only Look Once (object detection model)

2. Features to Be Tested

2.1 Perception Subsystem (fd_perception)

- Camera image input and YOLO model processing pipeline
- Lane detection and polyline extraction
- Obstacle classification and distance estimation
- Traffic sign recognition - STOP signs
- Combining ultrasonic and camera-based depth data
- Vehicle position estimation published at >= 1 Hz

2.2 Planning Subsystem (fd_planning)

- Road graph setup from Gazebo .sdf world file
- Dynamic blocking and unblocking of edges
- A* global path calculation visiting all target nodes within 10 s
- Automatic replanning when an edge is blocked
- Short-term trajectory generation (3 s window) with obstacle avoidance
- Energy cost evaluation of possible trajectories
- Behavior state changes for STOP sign, parking, and obstacle events

2.3 Control Subsystem (fd_control)

- PID-based steering and throttle/brake command generation
- Lateral and heading error calculation
- Emergency stop triggered when obstacle distance < 0.30 m
- Clearing the emergency state after the obstacle is removed
- Parking space detection using camera images
- Reverse parking planning and final position check

2.4 Simulation Interface (fd_simulation)

- Camera frame republishing with correct color format and timestamp
- Ultrasonic range limited to the valid sensor range [0.02 m, 4.0 m]
- Converting ControlCommand to AckermannDriveStamped
- Maximum brake command in EMSTOP mode
- Simulation control functions: reset, pause, unpause

2.5 End-to-End Scenarios

- Full driving cycle: Gazebo -> Perception -> Planning -> Control -> Gazebo
- Stop sign compliance scenario
- Emergency stop event scenario
- End-to-end parking maneuver
- Dynamic obstacle rerouting

2.6 Features NOT to Be Tested

- Gazebo physics engine internals -> treated as a trusted third-party simulator
- ROS2 DDS middleware layer -> tested by the ROS2 project's own test suite
- YOLO model training accuracy -> the model is used as a pre-trained component
- Hardware driver interfaces -> not included in the simulation-only release scope

3. Testing Methodology

The F&D testing strategy follows a four-level approach that matches the subsystem structure defined in the Low-Level Design Report. Tests are performed from the lowest level to the highest: unit tests check individual classes, integration tests verify communication between subsystems, system tests run full simulation scenarios, and performance tests ensure that non-functional requirements are met.

3.1 Unit Testing

Unit tests focus on individual classes and their public methods, tested in isolation. External dependencies (ROS2 nodes, Gazebo, YOLO model) are replaced with mocks or stubs.

- Framework -> pytest (Python 3.10+) with unittest.mock for ROS2 and sensor stubs.
- Each method’s pre-condition and post-condition, as defined in Section 3 of the LLD, is verified by at least one test case.
- Typed exceptions (PlanningTimeoutError, SensorFusionError, InsufficientDataError, NoValidPathError) are tested using invalid input scenarios.
- Coverage target -> at least 80% line coverage per package, measured with pytest-cov.

3.2 Integration Testing

Integration tests ensure that ROS2 topics and services connect subsystems correctly. Tests run in a minimal ROS2 launch setup without a live Gazebo instance; sensor data is replayed from ROS2 bag files.

- Perception to Planning -> ObstacleArray, LaneMarkings, and VehiclePose messages published by perception nodes are correctly used by BehaviorManager and LocalPlanner.
- Planning to Control -> GlobalPath and LocalTrajectory are received by TrajectoryTracker within the required latency.
- Control to Simulation -> ControlCommand messages are correctly converted into Gazebo Ackermann commands by GazeboActuatorBridge.
- Emergency override -> EmergencyStop output correctly overrides TrajectoryTracker commands.
- Framework -> rclpy test utilities with launch_testing for managing node lifecycles.

3.3 System Testing

System tests run full end-to-end scenarios in a live Gazebo simulation environment. These tests directly match the three scenarios shown in the LLD sequence diagram.

- Driving Cycle Test -> the vehicle successfully navigates through all goal nodes.
- Stop Sign Event Test -> the vehicle stops at a STOP sign for at least 3 seconds, then continues.
- Emergency Stop Event Test -> the vehicle fully brakes when an obstacle is placed within 0.30 m.
- Parking Maneuver Test -> the vehicle detects a parking space and parks within the required tolerance (< 0.15 m lateral error, < 20 deg heading error).
- Dynamic Replan Test -> when a road segment is blocked by an obstacle, the vehicle replans its path and reaches all goals.

3.4 Performance Testing

Performance tests evaluate the system against the non-functional requirements defined in the LLD.

- Vision pipeline throughput -> VisionProcessor.run_pipeline must maintain at least 10 FPS.
- Global planning latency -> GlobalPlanner.plan must complete within 10 seconds on the test map.
- Local planning update rate -> LocalPlanner.generate must produce a new trajectory every 3 seconds or less.
- Localization frequency -> LocalizationEstimator must publish data at least at 1 Hz.
- Depth estimation accuracy -> DepthEstimator.estimate must return distance values with no more than 20% error compared to ground truth.
- Tools -> Python time-it, ROS2 topic echo with timestamps, and custom benchmark scripts.

3.5 Regression Testing

Regression tests are a selected subset of unit and integration tests that are automatically run on every Git commit through a CI pipeline. Any test failure prevents merging into the main branch. The regression test suite is updated and expanded whenever a defect is fixed.

3.6 User Acceptance Testing (UAT)

UAT is carried out by the project supervisor and jury members based on the original functional requirements. Test scenarios are executed live in the Gazebo environment. Acceptance criterias:

- The vehicle autonomously visits all goal nodes without human intervention.
- The vehicle follows all STOP and PARK signs during the scenario.
- No collisions occur in any test run.
- The system responds to a dynamically added obstacle by replanning and successfully completing the mission.

4. Test Environment

Test Data

- Synthetic Gazebo worlds with predefined road networks, traffic signs, and parking bays.
- Recorded ROS2 packages capturing camera, ultrasonic, and pose data for regression replay.
- Ground-truth obstacle position logs exported from Gazebo for depth accuracy validation.
- A curated adversarial image set (occluded signs, poor lighting) for perception stress tests.

5. Test Cases

Naming convention -> [SUBSYSTEM]-[TYPE]-[NNN]. PER stands for Perception, PLN for Planning, CTL for Control, SIM for Simulation, INT for Integration, SYS for System, and PERF for Performance. U represents Unit.

5.1 Perception Unit Tests (PER-U)

Test ID	Test Name	Description	Input / Precondition	Expected Result
PER-U-001	VisionProcessor Init	Verify model loads successfully when valid YOLO weights path is provided.	Valid model_path; conf_threshold=0.5	Model loaded with no exceptions; run_pipeline is callable.
PER-U-002	VisionProcessor - Invalid Path	Verify appropriate exceptions raised on non-existent model paths.	Non-existent model_path string	FileNotFoundError or RuntimeError raised during __init__.
PER-U-003	Pipeline FPS	Verify vision pipeline sustains >= 10 FPS over 100 consecutive frames.	100 consecutive camera images frames fed to run_pipeline	Mean measured throughput >= 10 frames/s.
PER-U-004	detect_lanes - Valid	Verify LaneMarkings returned with left, right, and center polylines.	Image containing clear lane markings	LaneMarkings object returned with non-empty lists.
PER-U-005	detect_lanes - No Lanes	Verify InsufficientDataError raised when no lanes are detectable.	Blank or textureless image input	InsufficientDataError raised.
PER-U-006	ObstacleClassifier - Classify	Verify Obstacle returned with non-empty type and positive distance.	Detection with valid bounding box	Obstacle.type is non-empty; Obstacle.distance > 0.
PER-U-007	TrafficSignRecognizer - STOP	Verify STOP sign detected with confidence > 0.5.	Image frame containing a STOP sign	TrafficSign with type='STOP' in returned list; confidence > 0.5.
PER-U-008	ParkingBayDetection - PARK	Verify PARKING BAY detected with confidence > 0.5.	Image frame containing a PARKING BAY	“Parking Bay”' in the returned list; confidence > 0.5.
PER-U-009	DepthEstimator - Normal	Verify distance estimate within 20% of ground-truth distance.	Valid obstacle.bbox and camera image; known ground-truth distance	Returned float within +/- 20% of ground truth.
PER-U-010	Rear Ultrasonic Sensor	Verify Ultrasonic Sensor data when an object is behind and closer than 4 meters.	Distance data is not None	No errors occurred.

Test ID	Test Name	Description	Input / Precondition	Expected Result
PER-U-011	LocalizationEstimator - Frequency	Verify pose published at >= 1 Hz over a 5-second window.	gt_pose stream at 10 Hz for 5 s	At least 5 VehiclePose messages published on /perception/vehicle_pose .
PER-U-013	LocalizationEstimator - Reset	Verify state reset to initial_pose after reset() call.	Arbitrary current pose; initial_pose = (0,0,0)	get_pose() returns pose at origin after reset().

5.2 Planning Unit Tests (PLN-U)

Test ID	Test Name	Description	Input / Precondition	Expected Result
PLN-U-001	RoadGraph Init	Verify graph loads nodes and edges from valid world file.	Valid world file path	Graph is non-empty; all nodes and edges initialized with correct weights.
PLN-U-002	RoadGraph add_node	Verify nodes added with unique ID and correct pose.	Unique node_id; valid Point	Node retrievable via get_neighbors without exception.
PLN-U-003	RoadGraph add_edge	Verify directed edges created with correct distance and energy weight.	Two existing node IDs; distance=5.0; energy_weight=1.2	Edge returned in get_neighbors(from_id) with matching values.
PLN-U-004	RoadGraph block_edge	Verify blocked edge excluded from get_neighbors results.	Existing edge; block_edge() called	Blocked destination absent from get_neighbors output.
PLN-U-005	RoadGraph unblock_edge	Verify unblocked edge reappears in get_neighbors.	Previously blocked edge; unblock_edge() called	Destination reappears in get_neighbors output.
PLN-U-006	GlobalPlanner plan - Success	Verify GlobalPath returned visiting all goals within 10 s.	Valid start_pose; 3 goal nodes; loaded graph	GlobalPath is non-empty; computation time < 10 s.
PLN-U-007	GlobalPlanner plan - Timeout	Verify PlanningTimeoutError raised when computation exceeds 10 s.	Artificially constrained graph forcing > 10 s search	PlanningTimeoutError raised.
PLN-U-008	GlobalPlanner plan - No Path	Verify NoValidPathError raised when all routes are blocked.	All edges blocked in the graph	NoValidPathError raised.
PLN-U-009	GlobalPlanner rePlan	Verify new GlobalPath published within 10s after edge blockage.	Blocked edge event while vehicle is mid-route	New GlobalPath published; does not traverse the blocked edge.
PLN-U-010	LocalPlanner generate - Success	Verify LocalTrajectory returned within the 3s update cycle.	All inputs < 3000 ms old	LocalTrajectory returned; elapsed generation time <= 3s.
PLN-U-011	LocalPlanner generate - Stale Input	Verify InsufficientDataError raised on stale inputs.	obstacles timestamp > 3000 ms old	InsufficientDataError raised.

Test ID	Test Name	Description	Input / Precondition	Expected Result
PLN-U-012	BehaviorManager-STOP sign	Verify state transitions DRIVING -> STOPPING on nearby STOP signs.	State=DRIVING; on_sign_detected(STOP) called	get_state() returns STOPPING.
PLN-U-013	BehaviorManager - Resume after stop	Verify state transitions STOPPING -> DRIVING after 3 s timer.	State=STOPPING; 3 s elapsed	get_state() returns DRIVING.
PLN-U-014	BehaviorManager - Obstacle replan	Verify state transitions DRIVING -> REPLANNING on blocking obstacles.	State=DRIVING; on_obstacle_blocking() called	get_state() returns REPLANNING.
PLN-U-015	BehaviorManager - Task complete	Verify TASK_COMPLETE state when all goals are visited.	All goals marked via on_goal_reached()	get_state() returns TASK_COMPLETE.
PLN-U-016	BehaviorManager - Parking	Verify state transitions DRIVING -> PARKING on PARK sign.	State=DRIVING; on_sign_detected(PARK) called	get_state() returns PARKING.

5.3 Control Unit Tests (CTL-U)

Test ID	Test Name	Description	Input / Precondition	Expected Result
CTL-U-001	TrajectoryTracker - control_loop	Verify ControlCommand returned with all required fields in valid ranges.	Valid LocalTrajectory and VehiclePose	ControlCommand non-null; steering in [-1,1]; throttle and brake each in [0,1].
CTL-U-002	Lateral Error Computation	Verify lateral error computed correctly for a pose offset from trajectory.	Pose 0.5 m to the left of the nearest trajectory point	Returned lateral error == -0.5 m (signed convention).
CTL-U-003	Heading Error Computation	Verify heading error computed correctly for a yaw difference.	Pose yaw 0.3 rad greater than trajectory heading	Returned heading error == 0.3 rad.
CTL-U-004	PID Steering - Non-zero	Verify PID produces a non-zero steering angle for non-zero errors.	lat_err=0.2; head_err=0.1	Steering angle != 0.0 and within valid radian range.
CTL-U-005	EmergencyStop - Trigger	Verify True returned and EMSTOP published when obstacle is within 0.30 m.	Obstacle at 0.25 m; vehicle in motion	check_and_trigger() returns True; published ControlCommand mode = EMSTOP.
CTL-U-006	EmergencyStop - No Trigger	Verify False returned when closest obstacle is beyond 0.30 m.	Nearest obstacle at 0.50 m	check_and_trigger() returns False; no EMSTOP published.
CTL-U-007	EmergencyStop - Reset	Verify normal control resumes after obstacle clears and reset() is called.	Emergency state active; obstacle removed; reset() invoked	Subsequent check_and_trigger() returns False; ControlCommand mode = NORMAL.
CTL-U-008	ParkingController - Detect Bay	Verify ParkingBay returned an image containing blue bay markings.	Camera image with 0.15 m blue lines bordered by white	ParkingBay with valid entry_pose, width, and depth returned.
CTL-U-009	ParkingController - No Bay	Verify None returned when no parking bay is visible.	Camera image without parking markings	detect_bay() returns None.

Test ID	Test Name	Description	Input / Precondition	Expected Result
CTL-U-010	ParkingController - Plan Maneuver	Verify LocalTrajectory with reverse phases returned for valid bay.	Valid ParkingBay within 5 m; current VehiclePose	LocalTrajectory is non-empty and includes reverse motion segments.
CTL-U-011	ParkingController - Verify Parked (Pass)	Verify True when vehicle pose is within tolerance.	Pose within 0.15 m lateral and 10 deg heading of bay	verify_parked() returns True.
CTL-U-012	ParkingController - Verify Parked (Fail)	Verify False when vehicle pose exceeds tolerance.	Pose 0.30 m lateral from bay center	verify_parked() returns False.

5.4 Simulation Unit Tests (SIM-U)

Test ID	Test Name	Description	Input / Precondition	Expected Result
SIM-U-001	Ultrasonic Clip - High	Verify range clipped to 4.0 m when Gazebo reports > 4.0 m.	Range message value = 8.0 m	Published Range.range == 4.0 m.
SIM-U-002	ActuatorBridge - Normal	Verify valid AckermannDriveStamped published for a normal ControlCommand.	ControlCommand mode=NORMAL; steering=0.3; throttle=0.6	AckermannDriveStamped published with matching steering and throttle.
SIM-U-003	ActuatorBridge - EMSTOP	Verify maximum brake command published when mode is EMSTOP.	ControlCommand mode=EMSTOP	AckermannDriveStamped: throttle=0, brake=1.0 (maximum).
SIM-U-004	ActuatorBridge - Clamp	Verify steering angle clamped when value exceeds physical limits.	ControlCommand with steering outside valid range	translate_command() publishes clamped value within allowed bounds.
SIM-U-005	SimulationController - Reset	Verify reset_world() calls /reset_world service and returns True.	Gazebo simulation running	True returned; Gazebo world is reset to initial state.
SIM-U-006	Pause and Unpause	Verify pause() freezes sensor data and unpause() resumes it.	Simulation running normally	Sensor topics stop updating after pause(); resume after unpause().

5.5 Integration Tests (INT)

Test ID	Test Name	Description	Input / Precondition	Expected Result
INT-001	Perception -> Planning Topic Flow	Verify BehaviorManager receives ObstacleArray published by fd_perception.	Launch fd_perception and fd_planning; replay obstacle bag file	on_obstacle_blocking() invoked within 500 ms of ObstacleArray publication.
INT-002	Perception -> Control Pose Flow	Verify TrajectoryTracker receives VehiclePose from fd_perception.	Launch fd_perception and fd_control; replay pose bag file	TrajectoryTracker processes VehiclePose and outputs ControlCommand.
INT-003	Planning -> Control Trajectory Flow	Verify TrajectoryTracker receives LocalTrajectory from fd_planning.	Launch fd_planning and fd_control with mocked VehiclePose	ControlCommand published within 1 s of LocalTrajectory message arrival.
INT-004	Emergency Override Priority	Verify EmergencyStop ControlCommand overrides TrajectoryTracker output.	Both nodes active; obstacle inserted at 0.20 m	Final ControlCommand on actuator topic reflects EMSTOP, not TrajectoryTracker values.
INT-005	Control -> Simulation Bridge	Verify GazeboActuatorBridge translates ControlCommand to Gazebo topic.	ControlCommand published by fd_control	AckermannDriveStamped appears on Gazebo vehicle plugin topic within 100 ms.
INT-006	Sensor Bridge -> Perception	Verify fd_perception receives reformatted images from GazeboSensorBridge.	Raw Gazebo image on sensor topic	fd_perception/obstacles topic receives detections within 200 ms.
INT-007	RoadGraph Block -> GlobalPlanner Replan	Verify block_edge event triggers rePlan and a new path is published.	Running GlobalPlanner; block_edge() called mid-route	New GlobalPath published on /planning/global_path within 10 s.

5.6 System Tests (SYS)

Test ID	Test Name	Description	Input / Precondition	Expected Result
SYS-001	Driving Cycle - Full Route	Vehicle autonomously navigates to all 3 goal nodes without any collision.	Loaded Gazebo world; 3 goal nodes; clear road	All goals reached; no collision event; BehaviorManager state = TASK_COMPLETE.
SYS-002	Stop Sign Compliance	Vehicle stops at STOP sign for >= 3 s then resumes driving.	Gazebo scene with STOP sign on the planned route	Vehicle velocity == 0 for >= 3 s at sign location; then resumes to next goal.
SYS-003	Emergency Stop Scenario	Vehicle halts immediately when obstacle placed within 0.30 m ahead.	Vehicle in motion; obstacle inserted 0.20 m ahead via Gazebo plugin	Vehicle decelerates to 0 within one control cycle; EMSTOP command published.
SYS-004	Parking Maneuver	Vehicle detects bay and parks within required tolerance.	Gazebo scene with marked parking bay on route	verify_parked() returns True; BehaviorManager state = END_DRIVING.
SYS-005	Dynamic Replan	Vehicle replans and completes mission after mid-route road blockage.	Obstacle inserted blocking current path segment mid-mission	Vehicle takes alternative route; reaches all remaining goal nodes.
SYS-006	Multiple STOP Signs	Vehicle complies with two separate STOP signs on the same route.	Route containing two STOP signs at different locations	Vehicle stops >= 3 s at each sign; reaches final goal without collision.

5.7 Performance Tests (PERF)

Test ID	Test Name	Description	Input / Precondition	Expected Result
PERF-001	Vision Pipeline Throughput	Measure sustained FPS of run_pipeline over a 60-second run.	camera feed at 30 FPS input for 60 s	Mean FPS >= 10 over the full measurement window.
PERF-002	Global Planner Latency	Measure wall-clock time for plan() to complete on the 50-node test graph.	50-node RoadGraph; 3 goal nodes; 10 independent runs	Completion time <= 10 s on all 10 runs.
PERF-003	Local Planner Update Rate	Measure interval between consecutive LocalTrajectory publications.	Live simulation run for 120 s	Maximum interval between any two consecutive trajectories <= 3 s.
PERF-004	Localization Publish Rate	Measure /perception/vehicle_pose publication frequency over 30 s.	Simulation running for 30 s	Mean publish rate >= 1 Hz; no gap > 2 s between consecutive messages.
PERF-005	Depth Estimation Accuracy	Measure percentage error against Gazebo ground-truth distances.	50 obstacle samples at known distances ranging 0.3 m to 3.0 m	Mean absolute percentage error <= 20% across all samples.

6. Test Schedule

Phase	Start Date	End Date	Owner
Unit Test Development	20 Mar 2026	03 Apr 2026	All developers
Unit Test Execution	10 Apr 2026	17 Apr 2026	All developers
Integration Test Development	17 Apr 2026	20 Apr 2026	Can Soysal, Taha Ugan
Integration Test Execution	21 Apr 2026	23 Apr 2026	Can Soysal, Çağrı Şanlı
System Test Execution	24 Apr 2026	28 Apr 2026	Mustafa Cem Hür, Taha Ugan
Performance Test Execution	29 Apr 2026	30 Apr 2026	Taylan Kayalı
Defect Fix & Regression Run	01 May 2026	05 May 2026	All developers

7. Roles and Responsibilities

Name	Role	Responsibilities
Can Soysal	Test Lead / Perception Tester	Coordinates overall test plan while working on perceptions tests mainly.
Çağrı Şanlı	Planning Tester	Writes and executes planning tests.
Mustafa Cem Hür	Control & System Tester	Writes and executes control tests and all system tests.
Taha Ugan	Simulation & Integration Tester	Writes and executes simulation and integration tests.
Taylan Kayalı	Performance & Regression Tester	Owns all performance tests.

8. Control Procedures

8.1 Defect Management

All defects found during testing are recorded in the project issue tracker with the following details: test case ID, severity (Critical / Major / Minor / Trivial), description of observed vs. expected behavior, steps to reproduce, and the assigned person. Severity definitions:

- Critical -> system crash, failure of a safety feature (for example, EmergencyStop not activating), or any data loss.
- Major -> incorrect behavior that prevents a test case or scenario from passing, with no available workaround.
- Minor -> incorrect behavior with a workaround available; includes cosmetic or logging issues.
- Trivial -> style issues or low-impact formatting errors.

8.2 Test Execution and Logging

- Unit and integration tests are executed using pytest.
- System test outputs are saved as Gazebo packets along with a written pass/fail report.
- Performance results are recorded as CSV files, including run IDs and measured values.
- All test artifacts are stored in the repository under /tests/results/.

8.3 Pass/Fail Criteria

- A test case passes when all expected results in its specification are fully met.
- A test phase passes when all tests run and at least 95% of them pass.
- Performance tests pass when every metric meets or exceeds the defined threshold.
- UAT passes when all scenarios are completed with no open Critical or Major defects.

9. Risks and Mitigations

ID	Risk Description	Likelihood	Mitigation Strategy
R-01	ROS2 message latency on slower hardware causes timing-sensitive tests to fail intermittently.	Low	Add configurable tolerance parameters to timing assertions; document the hardware minimum requirements in Section 4.
R-02	Team member unavailability during the integration/system test phase reduces testing bandwidth.	Medium	Cross-train all members on each other's test suites; maintain a runbook for each test category.
R-03	GlobalPlanner A* exceeds 10 s timeout on adversarial road graphs.	Low	Test the graph for any infinity loops before taking any action.
R-04	EmergencyStop and TrajectoryTracker race condition under high CPU load produces incorrect ControlCommand ordering.	Low	Give high priority to emergency states and actions on threads and ROS2 nodes.

10. References

- IEEE Std 829-2008, IEEE Standard for Software and System Test Documentation.
- S. Macenski et al., 'Robot Operating System 2: Design, architecture, and uses in the wild,' Science Robotics, vol. 7, no. 66, May 2022.
- N. Koenig and A. Howard, 'Design and use paradigms for Gazebo,' IEEE/RSJ IROS, 2004.
- pytest Documentation — https://docs.pytest.org/
- ROS2 launch_testing Documentation — https://github.com/ros2/launch/tree/humble/launch_testing
- Ultralytics YOLOv8 Documentation — https://docs.ultralytics.com/